

NOA Sessions Footprints

// This Pine Script® code is subject to the terms of the Mozilla Public License 2.0 at <https://mozilla.org/MPL/2.0/>

// © nord-ouestadvisors

//@version=6

indicator("NOA Sessions Footprints", overlay = true, max_lines_count = 500, max_labels_count = 500, max_boxes_count = 500)

//-----

// Global Constants (Icons)

//-----

const string I_STK = "📊" // Stacked Initiative

const string I_SQZ = "🔴" // Delta Squeeze

const string I_EXH = "💀" // Exhaustion

const string I_DIV = "🔄" // Divergence

const string I_ABS = "🛡️" // Absorption

const string I_LRG = "⚡" // Large Candle

const string I_TRP = "🏠" // Trapped VA

const string I_CVD = "🔄" // CVD Flip

//-----

// General Settings

//-----

string modeInput = input×string("Intraday", "Indicator Mode", options = ["Intraday", "ETH", "RTH", "Both", "Full Day"], tooltip = "Select tracking mode.\n'Intraday': Tracks up to 4 custom time-based sessions.\n'ETH/RTH': Single session focus.\n'Both': Plots Extended Hours followed by Regular Trading Hours.\n'Full Day': Tracks the complete 24h market cycle.")

bool showSessionNames = input×bool(true, "Show session names", tooltip = "Displays the written text name of the session between the Unicode emoji heatmaps.")

bool showSummaryTable = input.bool(true, "Show Summary Table", tooltip = "Displays the Live vs. Historical order flow dashboard in the upper right corner.")

string linesToDraw = input×string("All", "Draw Session Lines", options = ["All", "High & Low", "Open & Close", "None"], tooltip = "Select which boundary lines to physically draw around the session bounds. (Does not affect background shading or Mid/VWAP lines).")

```

bool drawO      = linesToDraw == "All" or linesToDraw == "Open & Close"
bool drawH      = linesToDraw == "All" or linesToDraw == "High & Low"
bool drawL      = linesToDraw == "All" or linesToDraw == "High & Low"
bool drawC      = linesToDraw == "All" or linesToDraw == "Open & Close"

string midVwapChoice    = input.string("Mid", "Plot Mid/VWAP", options = ["Mid", "VWAP", "Both",
"None"], tooltip = "Plots internal moving averages.\nMid: Simple median of the session (High +
Low) / 2.\nVWAP: Volume Weighted Average Price for the session.")
int atrLength          = input.int(20, "Lookback Period (Days)", minval = 2, tooltip = "Number of
historical days used to calculate the 'Average Range', 'Average Volume', and 'Standard Deviation'
(Z-Scores) across all metrics.")
int fillTransparency    = input.int(85, "Fill Transparency", minval = 0, maxval = 100, tooltip =
"Adjusts the opacity of the green/red session background boxes. 0 = Solid paint, 100 = Completely
invisible.")
int maxHistorySessions  = input.int(20, "History Length", minval = 0, maxval = 100, tooltip =
"Maximum number of past sessions to keep drawn on the chart. Lower this number if your chart
begins to lag.")
string labelSizeInput   = input.string("Normal", "Text Size", options = ["Small", "Normal",
"Large"], tooltip = "Adjusts the physical size of the labels and emojis shown on the chart.")

float labelOffsetAtr     = input.float(0.0, "Base Offset (ATR)", minval = 0.0, step = 0.1, tooltip =
"Initial vertical distance from the Session Low to the first label. Uses ATR to scale automatically with
market volatility.")
float stackGapAtr        = input.float(0.75, "Stack Spacing (ATR)", minval = 0.0, step = 0.1, tooltip =
"Additional drop distance applied to overlapping sessions (the cascade step) so labels do not hide
each other.")

//-----
// Gap Visual Settings
//-----
const string GAP_GROUP   = "Gap Visual Settings"
bool plotGapBoxes        = input.bool(true, "Draw Gap Boxes & Lines", group = GAP_GROUP, tooltip =
"Toggles the visual gap boxes on the chart. (Gap statistics will still appear in session tooltips even
if this is off).")
int gapDuration          = input.int(26, "Plot Gap for N Bars", minval = 1, group = GAP_GROUP, tooltip =
"How many bars the physical gap box should extend to the right.")
bool showGapMid          = input.bool(true, "Show Mid-Gap Level Line", group = GAP_GROUP)
color gapUpColor         = input.color(color.new(color.green, 85), "Up Gap Fill", inline = "Up", group

```

```

= GAP_GROUP)
color gapUpBorder      = input.color(color.green, "Border", inline = "Up", group = GAP_GROUP)
color gapDnColor       = input.color(color.new(color.red, 85), "Down Gap Fill", inline = "Down",
group = GAP_GROUP)
color gapDnBorder      = input.color(color.red, "Border", inline = "Down", group = GAP_GROUP)
color gapMidLineCol    = input.xcolor(color.xnew(color.xgray, 50), "Mid-Gap Line Color", group =
GAP_GROUP)

//-----
// Alert Settings
//-----
const string ALERT_GROUP = "Alert Conditions"
bool alertStk           = input.bool(true, "Alert: Stacked Initiative (" + I_STK + ")", group =
ALERT_GROUP)
bool alertLrg           = input.bool(true, "Alert: Large Candle (" + I_LRG + ")", group =
ALERT_GROUP)
bool alertSqz           = input.bool(true, "Alert: Delta Squeeze (" + I_SQZ + ")", group =
ALERT_GROUP)
bool alertExh           = input.bool(true, "Alert: Exhaustion Print (" + I_EXH + ")", group =
ALERT_GROUP)
bool alertDiv           = input.bool(true, "Alert: Divergence (" + I_DIV + ")", group = ALERT_GROUP)
bool alertAbs           = input.bool(true, "Alert: Absorption (" + I_ABS + ")", group = ALERT_GROUP)
bool alertTrp           = input.bool(true, "Alert: Trapped VA (" + I_TRP + ")", group = ALERT_GROUP)
bool alertCvd           = input.bool(true, "Alert: CVD Flip (" + I_CVD + ")", group = ALERT_GROUP)
bool alertExpRng        = input.bool(true, "Alert: Expected Range Crossed", group =
ALERT_GROUP, tooltip = "Triggers when price trades outside the historical Expected Average Range
or the Z-Score Standard Deviation bands.")

//-----
// Statistical Probability Settings
//-----
const string PROB_GROUP = "Statistical Probability"
bool filterOutliers     = input.bool(true, "Exclude Outlier Extremes", group = PROB_GROUP, tooltip =
"Mathematically drops the top 5% and bottom 5% of historical data (excluding directional CVD).
Prevents massive news days from artificially inflating your expected ranges.")
string expRangeMode      = input.xstring("SDev", "Expected Range Basis", options = ["Average",
"SDev", "Both", "None"], group = PROB_GROUP, tooltip = "Projects expected boundaries.
\n'Average': Lines based on the historical mean range.\n'SDev': Lines based on historical Standard

```

Deviation limits.")

float sdMult = input×float(2.0, "SDev Multiplier", minval = 0.5, maxval = 5.0, step = 0.1, group = PROB_GROUP, tooltip = "Multiplier for the Standard Deviation bands (Spatial)×\n1.0 = ~68% probability zone×\n2.0 = ~95% probability zone.")

color avgRangeColor = input×color(color×new(#00FF00, 30), "Avg Range Color", group = PROB_GROUP, tooltip = "Color for the dashed lines mapping the historical average range.")

color sdRangeColor = input×color(color×new(#FF00FF, 30), "SDev Color", group = PROB_GROUP, tooltip = "Color for the dotted lines mapping the standard deviation volatility boundaries.")

//-----

// Footprint Signature Settings

//-----

const string GRP_ENGINE = "Footprint Engine Settings"

bool calcOnClose = input×bool(true, "Wait for Bar Close (No Flickering)", group = GRP_ENGINE, tooltip = "If true, signatures will only appear the moment the bar formally closes.")

int fpTicks = input×int(10, "Footprint Ticks per Row", group = GRP_ENGINE, tooltip = "Resolution for the volume footprint request. 10 is a standard institutional setting.")

int candleVaPercent = input.int(40, "Candle VA (%)", group = GRP_ENGINE, tooltip = "Percentage of volume used to calculate the internal Candle Value Area (VA).")

const string GRP_COLORS = "Master Signature Palettes"

color g1BullColor = input.color(color.new(#2e7d32, 0), "G1: Initiative Bull", inline = "g1", group = GRP_COLORS)

color g1BearColor = input.color(color.new(#8b0000, 0), "Bear", inline = "g1", group = GRP_COLORS)

color g2BullColor = input.color(color.new(#2962FF, 0), "G2: Reversals Bull", inline = "g2", group = GRP_COLORS)

color g2BearColor = input.color(color.new(#FF6D00, 0), "Bear", inline = "g2", group = GRP_COLORS)

color g3BullColor = input.color(color.new(#00FFFF, 0), "G3: Shifts/Traps Bull", inline = "g3", group = GRP_COLORS)

color g3BearColor = input.color(color.new(#FF007F, 0), "Bear", inline = "g3", group = GRP_COLORS)

const string GRP_SIG1 = "Signatures: Initiative (Group 1)"

bool showStacked = input.bool(true, "Mark Stacked Initiative (" + I_STK + ")", group = GRP_SIG1, tooltip = "Detects consecutive bars with aligned delta and price direction.")

```

int    stackedThreshold    = input.int(3, " └─ Stacked Bar Count", minval = 2, group = GRP_SIG1)
bool   reqStackedDelta     = input×bool(true, " └─ Require Min Delta %", inline = "stk_d", group =
GRP_SIG1)
float  stackedDeltaPct     = input.float(10.0, "", minval = 1.0, step = 1.0, inline = "stk_d", group =
GRP_SIG1)
bool   reqStackedDisplace  = input×bool(true, " └─ Require Streak Displacement", inline =
"stk_disp", group = GRP_SIG1)
float  stackedDisplaceMult = input.float(1.0, "(x Avg Range)", minval = 0.1, step = 0.1, inline =
"stk_disp", group = GRP_SIG1)

bool   showLargeCandles    = input.bool(true, "Mark Large Candles (" + I_LRG + ")", group =
GRP_SIG1, tooltip = "Detects explosive initiative via historically large candle bodies with aligned
order flow delta.")
float  candleSdMult        = input.float(2.0, " └─ Candle Z-Score", minval = 0.5, step = 0.1, inline =
"lrg", group = GRP_SIG1)
bool   reqAlignedDelta     = input×bool(true, "Require Aligned Delta", inline = "lrg", group =
GRP_SIG1)

const string GRP_SIG2      = "Signatures: Reversals & Absorption (Group 2)"
bool   showSqueeze         = input.bool(true, "Mark Delta Squeeze (" + I_SQZ + ")", group =
GRP_SIG2, tooltip = "Detects extreme offside aggression where overwhelming directional delta
results in a close in the opposite direction.")
float  sqzThreshold        = input.float(0.40, " └─ Min Delta % for Squeeze", minval = 0.1, step =
0.05, group = GRP_SIG2)

bool   showExhaustion      = input.bool(true, "Mark Exhaustion Print (" + I_EXH + ")", group =
GRP_SIG2, tooltip = "Detects climactic volume at a session extreme where delta aligns but price
rejects to close in the opposite half of the bar.")
float  exhVolZThreshold    = input.float(1.5, " └─ Vol Z-Score Threshold", minval = 0.5, step = 0.1,
group = GRP_SIG2)

bool   showDivergence      = input.bool(true, "Mark Divergence (" + I_DIV + ")", group = GRP_SIG2,
tooltip = "Detects exhaustion or liquidity sweeps. Triggers at session extremes when price takes out
a high/low but violently rejects with aggressive counter-trend order flow (Delta) closing outside the
Value Area.")

bool   showAbsorption      = input.bool(true, "Mark Absorption (" + I_ABS + ")", group = GRP_SIG2,
tooltip = "Detects limit-order walls (Icebergs). Triggers when massive volume is dumped into the

```

```

tape (High Effort) but the candle body remains tiny (Small Result) with opposing delta.")
float absVolSdMult      = input.float(1.5, " └─ Min Vol Z-Score (Effort)", inline = "abs", group =
GRP_SIG2)
float absBodyMaxMult    = input.float(0.5, "Max Body Ratio (Result)", inline = "abs", group =
GRP_SIG2)

const string GRP_SIG3    = "Signatures: Regime Shifts & Traps (Group 3)"
bool  showTrappedVA      = input.bool(true, "Mark Trapped VA (" + I_TRP + ")", group = GRP_SIG3,
tooltip = "Detects aggressive acceptance outside value. Triggers when price gaps directionally away
from the current candle's Value Area and drives heavily.")
float trapVaMultiplier   = input.float(1.0, " └─ Trap Gap Multiplier (x VA Range)", group =
GRP_SIG3, step = 0.5)

bool  showCvdFlip        = input.bool(true, "Mark CVD Flip (" + I_CVD + ")", group = GRP_SIG3,
tooltip = "Detects macro shifts in intraday control. Marks the exact candle where the Session's
Cumulative Volume Delta (CVD) crosses the zero line.")

//-----
// Special Line Colors
//-----
const string COLOR_GROUP = "Special Line Colors"
color  midLineColor      = input.color(color.new(color.yellow, 30), "Historical Mid", group =
COLOR_GROUP)
color  vwapLineColor     = input.color(color.new(color.orange, 0), "Historical VWAP", group =
COLOR_GROUP)
color  specLineColor     = input.color(color.red, "Active Mid/VWAP Override", group =
COLOR_GROUP)

//-----
// Extended & Intraday Mode Settings
//-----
bool  isExt              = modelInput != "Intraday"
const string GLOB_GROUP  = "Extended Mode Settings"
string globName          = input.string("Session", "Displayed Name", group = GLOB_GROUP, active
= isExt)
color  globColor         = input.color(color.new(color.gray, 50), "Session Color", group =
GLOB_GROUP, active = isExt)
string ethTime           = input.session("1800-0930", "ETH Time", group = GLOB_GROUP, active =

```

```
modelInput == "ETH" or modelInput == "Both")
string rthTime      = input.session("0930-1600", "RTH Time", group = GLOB_GROUP, active =
modelInput == "RTH" or modelInput == "Both")
```

```
const string HEAT_GROUP    = "Heatmap Thresholds"
float  atrExp              = input.float(120.0, "Red: Expansion (%)", group = HEAT_GROUP)
float  atrTar              = input.float(100.0, "Yellow: Target (%)", group = HEAT_GROUP)
float  atrNorm             = input.float(70.0, "White: Normal (%)", group = HEAT_GROUP)
```

```
bool  isIntra             = modelInput == "Intraday"
const string S1_G = "First", S2_G = "Second", S3_G = "Third", S4_G = "Fourth"
bool  shS1 = input.bool(true, "Show", group = S1_G, inline = "1", active = isIntra), string nS1 =
input.string("Tokyo", "", group = S1_G, inline = "1", active = isIntra), string tS1 =
input.session("0900-1530", "", group = S1_G, inline = "1", active = isIntra), string tzS1 =
input.string("Asia/Tokyo", "TZ", group = S1_G, active = isIntra), color cS1 =
input.color(color.new(#2962FF, 0), "", group = S1_G, inline = "1", active = isIntra)
bool  shS2 = input.bool(true, "Show", group = S2_G, inline = "2", active = isIntra), string nS2 =
input.string("London", "", group = S2_G, inline = "2", active = isIntra), string tS2 =
input.session("0800-1430", "", group = S2_G, inline = "2", active = isIntra), string tzS2 =
input.string("Europe/London", "TZ", group = S2_G, active = isIntra), color cS2 =
input.color(color.new(#FF9800, 0), "", group = S2_G, inline = "2", active = isIntra)
bool  shS3 = input.bool(true, "Show", group = S3_G, inline = "3", active = isIntra), string nS3 =
input.string("Atlantic", "", group = S3_G, inline = "3", active = isIntra), string tS3 =
input.session("0930-1130", "", group = S3_G, inline = "3", active = isIntra), string tzS3 =
input.string("America/New_York", "TZ", group = S3_G, active = isIntra), color cS3 =
input.color(color.new(#00BCD4, 0), "", group = S3_G, inline = "3", active = isIntra)
bool  shS4 = input.bool(true, "Show", group = S4_G, inline = "4", active = isIntra), string nS4 =
input.string("New York", "", group = S4_G, inline = "4", active = isIntra), string tS4 =
input.session("1130-1600", "", group = S4_G, inline = "4", active = isIntra), string tzS4 =
input.string("America/New_York", "TZ", group = S4_G, active = isIntra), color cS4 =
input.color(color.new(#E91E63, 0), "", group = S4_G, inline = "4", active = isIntra)
```

```
//-----
// Global State & Helpers
//-----
```

```
bool isNewDay = ta.change(time("D")) != 0
float gAtr = taxatr(atrLength)
```

```
footprint barFootprint = request×footprint(fpTicks, candleVaPercent)
float barDelta      = not na(barFootprint) ? barFootprint.delta() : na
float cVol          = not na(barFootprint) ? barFootprint.total_volume() : volume
```

```
volume_row g_vahRow  = not na(barFootprint) ? barFootprint.vah() : na
volume_row g_valRow  = not na(barFootprint) ? barFootprint.val() : na
volume_row g_pocRow  = not na(barFootprint) ? barFootprint.poc() : na
```

```
float globalVahPrice = not na(g_vahRow) ? g_vahRow.up_price() : na
float globalValPrice  = not na(g_valRow) ? g_valRow.down_price() : na
float globalPocPrice  = not na(g_pocRow) ? g_pocRow.up_price() : na
float globalVaRange   = (not na(globalVahPrice) and not na(globalValPrice)) ?
math.max(syminfo.mintick, globalVahPrice - globalValPrice) : na
```

```
float prevCVol      = cVol[1]
float prevHigh      = high[1]
float prevLow       = low[1]
float prevClose     = close[1]
float prevPoc1      = globalPocPrice[1]
```

```
int curBullStreak = 0
int curBearStreak = 0
```

```
float barDeltaPct = cVol > 0 ? (barDelta / cVol) * 100.0 : 0
bool passBullDelta = not reqStackedDelta or (barDeltaPct >= stackedDeltaPct)
bool passBearDelta = not reqStackedDelta or (barDeltaPct <= -stackedDeltaPct)
```

```
bool isBullStep = (barDelta > 0) and (close > open) and (close > prevClose) and (globalPocPrice >
prevPoc1) and passBullDelta
bool isBearStep = (barDelta < 0) and (close < open) and (close < prevClose) and (globalPocPrice <
prevPoc1) and passBearDelta
```

```
curBullStreak := isBullStep ? nz(curBullStreak[1]) + 1 : 0
curBearStreak := isBearStep ? nz(curBearStreak[1]) + 1 : 0
```

```
// DRY Helpers
```

```
toTicks(float x) => int(math.round(x / syminfo.mintick))
```

```
getZ(float cur, float avg, float sd) => (not na(avg) and not na(sd) and sd > 0) ? (cur - avg) / sd : na
```



```
method add(table t, int c, int r, string txt, color col = color.white, string align = text.align_left) =>
table.cell(t, c, r, txt, text_color = col, text_size = size.small, text_halign = align)
```

```
fireAlert(bool cond, bool toggle, string icon, string name, bool isBull, string title) =>
  if cond and toggle
    string dir = isBull ? "Bullish" : "Bearish"
    alert(icon + " " + name + ": " + dir + " " + title + " detected on " + syminfo.ticker,
    alert.freq_once_per_bar)
```

```
getHlcon(float cur, float avg) =>
  if na(avg) or avg == 0
    "❄️"
  else
    float r = (cur / avg) * 100.0
    r >= atrExp ? "🔥" : r >= atrTar ? "🌡️" : r >= atrNorm ? "☁️" : "❄️"
```

```
getCvdIcon(float cvd, float vol) =>
  if na(cvd) or na(vol) or vol == 0
    "❄️"
  else
    float agg = math.abs((cvd / vol) * 100.0)
    agg >= 15.0 ? "🔥" : agg >= 5.0 ? "🌡️" : agg >= 2.0 ? "☁️" : "❄️"
```

```
getLSize(string s) => s == "Small" ? size.small : s == "Normal" ? size.normal : size.large
```

```
fN(float x) => str.format("{0,number,###}", x)
```

```
fP(float x) => na(x) ? "" : str.format("({0,number,###}%)", x * 100.0)
```

```
fV(float v) =>
```

```
  float vA = math.abs(v)
  vA >= 1e9 ? str.format("{0,number,###}B", v/1e9) : vA >= 1e6 ? str.format("{0,number,###}M", v/
1e6) : vA >= 1e3 ? str.format("{0,number,###}K", v/1e3) : str.format("{0,number,0}", v)
```

```
var gapHistory = array.new<float>()
```

```
var string currentDayGapIcon = ""
```

```
var string currentDayGapTooltip = ""
```

```
var float globalAvgGap = na
```

```
var float globalTodayGap = na
```

```
var gapBoxes = array.new<box>()
```

```
var gapLines = array.new<line>()
```

```
//-----
```

```
// Types
```

```
//-----
```

```
type SessionDisplay
```

```
    label    infoLab
```

```
    box      bgBox
```

```
    line     oLin
```

```
    line     hLin
```

```
    line     lLin
```

```
    line     mLin
```

```
    line     vLin
```

```
    line     cLin
```

```
    line     avgHLin
```

```
    line     avgLLin
```

```
    line     sdHLin
```

```
    line     sdLLin
```

```
    float    oPri
```

```
    float    hPri
```

```
    float    lPri
```

```
    float    cPri
```

```
    float    vSum
```

```
    float    pvSum
```

```
    float    pv2Sum
```

```
    int      sIdx
```

```
    float    avgR
```

```
    float    avgV
```

```
    float    avgSBody
```

```
    float    stdevR
```

```
    float    stdevV
```

```
    float    stdevSBody
```

```
    float    avgBarRange
```

```
    float    sdBarRange
```

```
    float    avgBody
```

```
    float    sdBody
```

```
    float    avgBarVol
```

```
    float    sdBarVol
```

```
float      cvd
array<label> sigLabels
bool       hitAvgH
bool       hitAvgL
bool       hitSdH
bool       hitSdL
```

```
type SessionState
```

```
array<float>      hR
array<float>      hV
array<float>      hCvd
array<float>      hSBody
array<float>      hBarRange
array<float>      hBody
array<float>      hBarVol
array<SessionDisplay> history
SessionDisplay    active
array<int>         hAvgHits
array<int>         hSdHits
```

```
initSessionState() =>
```

```
    SessionState.new(array.new<float>(), array.new<float>(), array.new<float>(), array.new<float>(),
array.new<float>(), array.new<float>(), array.new<float>(), array.new<SessionDisplay>(), na,
array.new<int>(), array.new<int>())
```

```
//-----
```

```
// Methods
```

```
//-----
```

```
method excludeExtremes(array<float> this) =>
```

```
    int sz = array.size(this)
```

```
    if not filterOutliers or sz < 10
```

```
        this
```

```
    else
```

```
        int trimCount = math×max(1, int(math.round(sz * 0.05)))
```

```
        array<float> copyArr = array.copy(this)
```

```
        array.sort(copyArr)
```

```
        for i = 1 to trimCount
```

```
            array.pop(copyArr)
```

```
array.shift(copyArr)
copyArr
```

```
method getAverages(SessionState this) =>
```

```
array<float> fR = this.hR.excludeExtremes()
array<float> fV = this.xhV.excludeExtremes()
array<float> fCvd = this.hCvd
array<float> fbR = this.hBarRange.excludeExtremes()
array<float> fbB = this.xhBody.excludeExtremes()
array<float> fbV = this.xhBarVol.excludeExtremes()
```

```
float a_sR = array.size(fR) > 0 ? array.avg(fR) : na
float a_sRPct = not na(a_sR) and close > 0 ? (a_sR / close) * 100.0 : na
float a_sV = array.size(fV) > 0 ? array.avg(fV) : na
float a_sCvd = array.size(fCvd) > 0 ? array.avg(fCvd) : na
float a_sCvdPct = not na(a_sCvd) and not na(a_sV) and a_sV > 0 ? (a_sCvd / a_sV) * 100.0 : na
```

```
float a_bR = array.size(fbR) > 0 ? array.avg(fbR) : na
float a_bB = array.size(fbB) > 0 ? array.avg(fbB) : na
float a_bV = array.size(fbV) > 0 ? array.avg(fbV) : na
```

```
string sR_str = not na(a_sR) ? str.toString(toTicks(a_sR)) : "-"
string sRPct_str = not na(a_sRPct) ? str.format("{0,number,###}%", a_sRPct) : "-"
string sV_str = not na(a_sV) ? fV(a_sV) : "-"
string sCvd_str = not na(a_sCvd) ? fV(a_sCvd) : "-"
string sCvdPct_str = not na(a_sCvdPct) ? str.format("{0,number,###}%", a_sCvdPct) : "-"
```

```
string bR_str = not na(a_bR) ? str.toString(toTicks(a_bR)) : "-"
string bB_str = not na(a_bB) ? str.toString(toTicks(a_bB)) : "-"
string bV_str = not na(a_bV) ? fV(a_bV) : "-"
```

```
int totalAvgHits = array.sum(this.hAvgHits)
int totalSdHits = array.sum(this.hSdHits)
int histCount = array.size(this.hAvgHits)
```

```
string avgHitStr = (expRangeMode == "Average" or expRangeMode == "Both") ?
str.toString(totalAvgHits) + "/" + str.toString(histCount) : "-"
string sdHitStr = (expRangeMode == "SDev" or expRangeMode == "Both") ?
```

```
str.toString(totalSdHits) + "/" + str.toString(histCount) : "-"
```

```
[sR_str, sRPct_str, sV_str, bR_str, bB_str, bV_str, sCvd_str, sCvdPct_str, avgHitStr, sdHitStr]
```

```
method getLiveStats(SessionState this) =>
```

```
string c_sR = "-"
```

```
string c_sRPct = "-"
```

```
string c_sV = "-"
```

```
string c_bR = "-"
```

```
string c_bB = "-"
```

```
string c_bV = "-"
```

```
string c_sCvd = "-"
```

```
string c_sCvdPct = "-"
```

```
string curAvgHitStr = "-"
```

```
string curSdHitStr = "-"
```

```
if not na(this.active)
```

```
float pR = this×active×hPri - this×active×lPri
```

```
c_sR := str.toString(toTicks(pR))
```

```
if this.active.oPri > 0
```

```
    c_sRPct := str.format("{0,number,###}%", (pR / this.active.oPri) * 100.0)
```

```
c_sV := fV(this.active.vSum)
```

```
c_sCvd := fV(this.active.cvd)
```

```
if this.active.vSum > 0
```

```
    c_sCvdPct := str.format("{0,number,###}%", (this.active.cvd / this.active.vSum) * 100.0)
```

```
c_bR := str.toString(toTicks(high - low))
```

```
c_bB := str.toString(toTicks(math.abs(close - open)))
```

```
c_bV := fV(cVol)
```

```
if expRangeMode == "Average" or expRangeMode == "Both"
```

```
    if this.active.hitAvgH and this.active.hitAvgL
```

```
        curAvgHitStr := "H+L"
```

```
    else if this.active.hitAvgH
```

```
        curAvgHitStr := "H"
```

```
    else if this.active.hitAvgL
```

```
        curAvgHitStr := "L"
```

```
else
    curAvgHitStr := "—"
```

```
if expRangeMode == "SDev" or expRangeMode == "Both"
    if this.active.hitSdH and this.active.hitSdL
        curSdHitStr := "H+L"
    else if this.active.hitSdH
        curSdHitStr := "H"
    else if this.active.hitSdL
        curSdHitStr := "L"
    else
        curSdHitStr := "—"
```

```
[c_sR, c_sRPct, c_sV, c_bR, c_bB, c_bV, c_sCvd, c_sCvdPct, curAvgHitStr, curSdHitStr]
```

```
method destroy(SessionDisplay this) =>
```

```
label.delete(this.infoLab), box.delete(this.bgBox)
line.delete(this.oLin), line.delete(this.hLin), line.delete(this.lLin)
line.delete(this.mLin), line.delete(this.vLin), line.delete(this.cLin)
line.delete(this.avgHLin), line.delete(this.avgLLin)
line.delete(this.sdHLin), line.delete(this.sdLLin)
```

```
if not na(this.sigLabels)
    for lbl in this.sigLabels
        label.delete(lbl)
```

```
method refreshLabels(SessionDisplay this, color baseC, string name, bool isActive, int stackLvl) =>
```

```
float sOpen  = not na(this.oPri) and this.oPri != 0.0 ? this.oPri : na
float sClose = this.cPri
float pR     = this.hPri - this.lPri
float delta  = sClose - this.oPri
float vwapV  = this.vSum > 0.0 ? this.pvSum / this.vSum : na
```

```
color mCol = isActive ? specLineColor : midLineColor
color vLCol = isActive ? specLineColor : vwapLineColor
color dirColor = sClose >= this.oPri ? color.green : color.red
```

```
if not na(this.mLin)
```

```

    line.set_color(this.mLin, mCol)
if not na(this.vLin)
    line.set_color(this.vLin, vLCol)

float eHigh = not na(this.avgR) ? this.oPri + (this.avgR / 2.0) : na
float eLow  = not na(this.avgR) ? this.oPri - (this.avgR / 2.0) : na
float sdHigh = not na(this.stdevR) ? this.oPri + ((this.stdevR * sdMult) / 2.0) : na
float sdLow  = not na(this.stdevR) ? this.oPri - ((this.stdevR * sdMult) / 2.0) : na

string sepPrice = "——— Price & Volatility ——-\n"
string t1 = sepPrice

int pRTicks = toTicks(pR)
string pR_pct = not na(sOpen) ? fP(pR/sOpen) : ""
t1 += "Range: " + str.toString(pRTicks) + " Ticks" + pR_pct + "\n"

if not na(this.avgR)
    int avgRTicks = toTicks(this.avgR)
    string avg_pct = not na(sOpen) ? fP(this.avgR/sOpen) : ""
    t1 += "Avg Range: " + str.toString(avgRTicks) + " Ticks" + avg_pct + "\n"

int moveTicks = toTicks(delta)
string move_pct = not na(sOpen) ? fP(delta/sOpen) : ""
t1 += "Net Move: " + (moveTicks > 0 ? "+" : "") + str.toString(moveTicks) + " Ticks" + move_pct +
"\n"

t1 += "VWAP: " + fN(vwapV) + "\n"

float vwapVariance = (this.xvSum > 0) ? (this.pv2Sum / this.vSum) - (vwapV * vwapV) : 0
float trueVwapSD = (vwapVariance > 0 and bar_index > this.sldx) ? math.sqrt(vwapVariance) : na
float sdDist = not na(trueVwapSD) and trueVwapSD > 0 ? (sClose - vwapV) / trueVwapSD : na
string sdDistStr = not na(sdDist) ? " (" + (sdDist > 0 ? "+" : "") + fN(sdDist) + " SD from VWAP)" :
""

t1 += "Last: " + fN(sClose) + sdDistStr

t1 += "\n\n——— Volume & Flow ——-\n"
string volStr = fV(this.xvSum)

```

```

if not na(this.avgV) and this.avgV > 0
    volStr += " (" + fN(this.vSum/this.avgV) + "x Avg)"
t1 += "Volume: " + volStr + "\n"

float volZ = getZ(this.vSum, this.avgV, this.stdevV)
if not na(volZ)
    t1 += "Vol Z-Score: " + (volZ > 0 ? "+" : "") + fN(volZ) + "\n"

float cvdRatio = this.vSum > 0 ? (this.cvd / this.vSum) * 100.0 : na
string cvdRatioStr = not na(cvdRatio) ? " (" + (cvdRatio > 0 ? "+" : "") + fN(cvdRatio) + "%)" : ""
t1 += "Session CVD: " + fV(this.cvd) + cvdRatioStr

bool showExp = (expRangeMode == "Average" or expRangeMode == "Both") and not na(eHigh)
bool showSD = (expRangeMode == "SDev" or expRangeMode == "Both") and not na(sdHigh)

if showExp or showSD
    t1 += "\n\n———— Projections ————\n"
    if showExp
        t1 += "Expected Range (Symmetric)\n High: " + fN(eHigh) + "\n Low: " + fN(eLow)
    if showSD
        t1 += (showExp ? "\n" : "") + "Volatility Bands (" + fN(sdMult) + " SD)\n High: " + fN(sdHigh)
+ "\n Low: " + fN(sdLow)

string rnglcon = getHlcon(pR, this.avgR)
string vollcon = getHlcon(this.vSum, this.avgV)
string cvdlcon = getCvdlcon(this.cvd, this.vSum)

// --- MACRO SESSION PROFILE ---
string macrolcon = ""
string macroText = ""
if not isActive and not na(this.avgSBody) and not na(this.stdevSBody) and not na(this.avgV) and
not na(this.stdevV)
    float sBody = mathxabs(sClose - sOpen)
    float sMid = (this.hPri + this.lPri) / 2.0

    bool m_largeBull = sBody >= (this.avgSBody + this.stdevSBody * candleSdMult) and (not
reqAlignedDelta or this.cvd > 0) and sClose > sOpen
    bool m_largeBear = sBody >= (this.avgSBody + this.stdevSBody * candleSdMult) and (not

```


reqAlignedDelta or this.cvd < 0) and sClose < sOpen

bool m_highEffortAbs = this.vSum > (this.avgV + this.stdevV * absVolSdMult)

bool m_lowResult = sBody < (this.avgSBody * absBodyMaxMult)

bool m_absBull = m_highEffortAbs and m_lowResult and this.cvd < 0

bool m_absBear = m_highEffortAbs and m_lowResult and this.cvd > 0

bool m_highVolExh = this.vSum > (this.avgV + this.stdevV * exhVolZThreshold)

bool m_exhBull = m_highVolExh and not m_lowResult and this.cvd < 0 and sClose > sMid

bool m_exhBear = m_highVolExh and not m_lowResult and this.cvd > 0 and sClose < sMid

if m_largeBull or m_largeBear

 macrolcon := " [" + I_LRG + "]"

 macroText := "Trend Session (Large Body)"

else if m_exhBull or m_exhBear

 macrolcon := " [" + I_EXH + "]"

 macroText := "Reversal Session (Exhaustion)"

else if m_absBull or m_absBear

 macrolcon := " [" + I_ABS + "]"

 macroText := "Rotational Session (Absorption)"

if macroText != ""

 t1 += "\n\n——— Macro Profile ———\nType: " + macroText

string gapPrefix = currentDayGaplcon != "" ? currentDayGaplcon + " " : ""

string decoratedName = gapPrefix + rnglcon + " " + name + " " + vollcon + " " + cvdlcon +
macrolcon

string compactName = gapPrefix + rnglcon + "  " + vollcon + " " + cvdlcon + macrolcon

if currentDayGapTooltip != ""

 t1 += currentDayGapTooltip

int midIdx = int(math.round((this.sldx + bar_index) * 0.5))

string outName = showSessionNames ? decoratedName : compactName

float yLoc = this.IPri - (nz(gAtr, 0) * labelOffsetAtr) - (nz(gAtr, 0) * stackGapAtr * stackLvl)

label.set_text(this.infoLab, outName)

```
label.set_tooltip(this.infoLab, t1)
label.set_x(this.infoLab, midIdx)
label.set_y(this.infoLab, yLoc)
label.set_style(this.infoLab, label.style_none)
label.set_textcolor(this.infoLab, dirColor)
```

```
if (expRangeMode == "Average" or expRangeMode == "Both") and not na(eHigh)
    line.set_x2(this.avgHLLin, bar_index), line.set_y1(this.avgHLLin, eHigh), line.set_y2(this.avgHLLin,
eHigh)
    line.set_x2(this.avgLLin, bar_index), line.set_y1(this.avgLLin, eLow), line.set_y2(this.avgLLin,
eLow)
if (expRangeMode == "SDev" or expRangeMode == "Both") and not na(sdHigh)
    line.set_x2(this.sdHLLin, bar_index), line.set_y1(this.sdHLLin, sdHigh), line.set_y2(this.sdHLLin,
sdHigh)
    line.set_x2(this.sdLLin, bar_index), line.set_y1(this.sdLLin, sdLow), line.set_y2(this.sdLLin,
sdLow)
```

// Extracted Method: Dynamic Multi-Signature System with Alerts

```
method evaluateSignatures(SessionDisplay d, float prevCvd, string name) =>
    float cBody = math×abs(close - open)
    float cRange = high - low
    bool isBull = close >= open

    float volPctChg = prevCVol > 0 ? ((cVol - prevCVol) / prevCVol) * 100.0 : na
    float deltaPct = not na(barDelta) and cVol > 0 ? (barDelta / cVol) * 100.0 : na

    float rangeZScore = getZ(cRange, d.avgBarRange, d.sdBarRange)
    float bodyZScore = getZ(cBody, d.avgBody, d.sdBody)
    float volZScore = getZ(cVol, d.avgBarVol, d.sdBarVol)

    int cRangeTicks = toTicks(cRange)
    int cBodyTicks = toTicks(cBody)

    float vahPrice = globalVahPrice
    float valPrice = globalValPrice
    float vaRange = globalVaRange
```

```
string vaString = (not na(valPrice) ? fN(valPrice) : "N/A") + " - " + (not na(vahPrice) ?
```

```

fN(vahPrice) : "N/A")
    string vaProfileText = ""
    string controlText = ""

    if not na(vahPrice) and not na(valPrice)
        if close > vahPrice
            float excess = close - vahPrice
            int excessTicks = toTicks(excess)
            string excessPctStr = cRange > 0 ? fN((excess / cRange) * 100) + "%" : "0%"

            int gapTicks = toTicks(open - vahPrice)
            string gapMultStr = (not na(vaRange) and vaRange > 0) ? fN((open - vahPrice) / vaRange) :
"0"

            string gapStr = gapTicks >= 0 ? "\nVA Gap: +" + str.toString(gapTicks) + " Ticks (" +
gapMultStr + "x VA)" : ""

            vaProfileText := ", Bullish Close"
            controlText := "Control: +" + str.toString(excessTicks) + " Ticks (" + excessPctStr + ")" +
gapStr + "\n"

        else if close < valPrice
            float excess = valPrice - close
            int excessTicks = toTicks(excess)
            string excessPctStr = cRange > 0 ? fN((excess / cRange) * 100) + "%" : "0%"

            int gapTicks = toTicks(valPrice - open)
            string gapMultStr = (not na(vaRange) and vaRange > 0) ? fN((valPrice - open) / vaRange) :
"0"

            string gapStr = gapTicks >= 0 ? "\nVA Gap: +" + str.toString(gapTicks) + " Ticks (" +
gapMultStr + "x VA)" : ""

            vaProfileText := ", Bearish Close"
            controlText := "Control: -" + str.toString(excessTicks) + " Ticks (" + excessPctStr + ")" +
gapStr + "\n"

        else
            vaProfileText := ", Neutral Close"
            controlText := "Control: Inside VA\n"

```

```
float candleSD = getZ(close - open, 0, dxsdBody) // Directional SD logic preserved
string cSdSign = candleSD > 0 ? "+" : ""
string dirSdStr = not na(candleSD) ? "\nDirectional SD: " + cSdSign + fN(candleSD) : ""
```

```
string rangeMultStr = not na(dxavgBarRange) and d.avgBarRange > 0 ? " (" + fN(cRange /
d.avgBarRange) + "x Avg)" : ""
string bodyMultStr = not na(dxavgBody) and d.avgBody > 0 ? " (" + fN(cBody / d.avgBody) + "x
Avg)" : ""
string volMultStr = not na(dxavgBarVol) and d.avgBarVol > 0 ? " (" + fN(cVol / d.avgBarVol) + "x
Avg)" : ""
```

```
float cvdRatio = d.vSum > 0 ? (d.cvd / d.vSum) * 100.0 : na
string cvdRatioStr = not na(cvdRatio) ? " (" + (cvdRatio > 0 ? "+" : "") + fN(cvdRatio) + "%)" : ""
```

```
string baseTip = "———— Price & Volatility ————\n"
baseTip += "Value Area: " + vaString + "\n"
baseTip += "Last: " + fN(close) + vaProfileText + "\n"
if controlText != ""
    baseTip += controlText
```

```
baseTip += "\nRange: " + str.toString(cRangeTicks) + " Ticks" + rangeMultStr + "\n"
baseTip += "Range Z-Score: " + (rangeZScore > 0 ? "+" : "") + fN(rangeZScore) + "\n"
```

```
baseTip += "\nBody: " + str.toString(cBodyTicks) + " Ticks" + bodyMultStr + "\n"
baseTip += "Abs Z-Score: " + (bodyZScore > 0 ? "+" : "") + fN(bodyZScore) + dirSdStr + "\n"
```

```
baseTip += "\n———— Volume & Flow ————\n"
baseTip += "Volume: " + fV(cVol) + volMultStr + "\n"
baseTip += "Volume %: " + (volPctChg > 0 ? "+" : "") + fN(volPctChg) + "%\n"
baseTip += "Vol Z-Score: " + (volZScore > 0 ? "+" : "") + fN(volZScore) + "\n\n"
```

```
baseTip += "Delta: " + fV(barDelta) + "\n"
baseTip += "Delta %: " + (deltaPct > 0 ? "+" : "") + fN(deltaPct) + "%\n"
baseTip += "Session Vol: " + fV(d.vSum) + "\n"
baseTip += "Session CVD: " + fV(d.cvd) + cvdRatioStr
```

```
// --- Signatures ---
```

```
bool isSweepLow = (bar_index > d.sldx + 2) and (low <= d.lPri)
```

```
bool isSweepHigh = (bar_index > d.sldx + 2) and (high >= d.hPri)
```

```
bool sqzBull = false, sqzBear = false
```

```
if showSqueeze and cVol > 0 and not na(barDelta)
```

```
    sqzBull := (barDelta <= -cVol * sqzThreshold) and (close > open)
```

```
    sqzBear := (barDelta >= cVol * sqzThreshold) and (close < open)
```

```
bool divBull = false, divBear = false
```

```
if showDivergence and not na(barDelta)
```

```
    divBull := isSweepLow and (close > open) and (barDelta > 0) and (not na(vahPrice) and close > vahPrice)
```

```
    divBear := isSweepHigh and (close < open) and (barDelta < 0) and (not na(valPrice) and close < valPrice)
```

```
bool exhBull = false, exhBear = false
```

```
if showExhaustion and not na(volZScore) and not na(barDelta)
```

```
    float midBar = (high + low) / 2.0
```

```
    exhBull := isSweepLow and (volZScore >= exhVolZThreshold) and (barDelta < 0) and (close > midBar)
```

```
    exhBear := isSweepHigh and (volZScore >= exhVolZThreshold) and (barDelta > 0) and (close < midBar)
```

```
// --- Stacked Initiative with Displacement Filter ---
```

```
bool stkBull = false, stkBear = false
```

```
if showStacked and (bar_index >= d.sldx + stackedThreshold - 1)
```

```
    bool streakBullOk = curBullStreak >= stackedThreshold
```

```
    bool streakBearOk = curBearStreak >= stackedThreshold
```

```
if streakBullOk or streakBearOk
```

```
    float streakOpen = open[stackedThreshold - 1]
```

```
    float totalMove = math×abs(close - streakOpen)
```

```
    float reqMove = reqStackedDisplace and not na(d.avgBarRange) ? (d.avgBarRange * stackedDisplaceMult) : 0
```

```
if streakBullOk and (totalMove >= reqMove)
```

```
    stkBull := true
```

```
if streakBearOk and (totalMove >= reqMove)
```

```
    stkBear := true
```

```

bool absBull = false, absBear = false
if showAbsorption and not na(d.avgBarVol) and not na(d.sdBarVol) and not na(d.avgBody) and
not na(barDelta)
    bool highEffort = cVol > (d.avgBarVol + d.sdBarVol * absVolSdMult)
    bool lowResult = cBody < (d.avgBody * absBodyMaxMult)
    absBull := highEffort and lowResult and barDelta < 0
    absBear := highEffort and lowResult and barDelta > 0

bool isLargeBull = false, isLargeBear = false
if showLargeCandles and not na(d.avgBody) and not na(d.sdBody)
    bool deltaOk = not reqAlignedDelta or (isBull ? (barDelta > 0) : (barDelta < 0))
    bool isLarge = cBody >= (d.avgBody + d.sdBody * candleSdMult) and deltaOk
    isLargeBull := isLarge and isBull
    isLargeBear := isLarge and not isBull

// --- Trapped VA ---
bool trappedBull = false, trappedBear = false
if showTrappedVA and not na(barDelta) and not na(vaRange)
    trappedBull := (not na(vahPrice) and open - vahPrice >= (vaRange * trapVaMultiplier)) and
(close > open) and (barDelta > 0)
    trappedBear := (not na(valPrice) and valPrice - open >= (vaRange * trapVaMultiplier)) and
(close < open) and (barDelta < 0)

bool cvdBull = false, cvdBear = false
if showCvdFlip and not na(barDelta)
    cvdBull := prevCvd <= 0 and d.cvd > 0
    cvdBear := prevCvd >= 0 and d.cvd < 0

// --- Intrabar Repaint Gate ---
bool isReady = calcOnClose ? barstate.isconfirmed : true

bool hasStk = (stkBull or stkBear) and isReady
bool hasSqz = (sqzBull or sqzBear) and isReady
bool hasExh = (exhBull or exhBear) and isReady
bool hasDiv = (divBull or divBear) and isReady
bool hasAbs = (absBull or absBear) and isReady
bool hasLrg = (isLargeBull or isLargeBear) and isReady

```

bool hasTrp = (trappedBull or trappedBear) and isReady

bool hasCvd = (cvdBull or cvdBear) and isReady

if barstate.isconfirmed

fireAlert(hasStk, alertStk, I_STK, name, stkBull, "Stacked Initiative")

fireAlert(hasSqz, alertSqz, I_SQZ, name, sqzBull, "Delta Squeeze")

fireAlert(hasExh, alertExh, I_EXH, name, exhBull, "Exhaustion Print")

fireAlert(hasLrg, alertLrg, I_LRG, name, isLargeBull, "Large Body")

fireAlert(hasDiv, alertDiv, I_DIV, name, divBull, "Divergence")

fireAlert(hasAbs, alertAbs, I_ABS, name, absBull, "Absorption")

fireAlert(hasTrp, alertTrp, I_TRP, name, trappedBull, "Trapped VA")

fireAlert(hasCvd, alertCvd, I_CVD, name, cvdBull, "CVD Flip")

if hasStk or hasSqz or hasExh or hasDiv or hasAbs or hasLrg or hasTrp or hasCvd

string iconString = ""

string titleString = ""

string descString = ""

color mainColor = color.gray

if hasStk

iconString += I_STK

titleString += (titleString == "" ? "" : ", ") + "Stacked Initiative"

descString += (descString == "" ? "" : "\n") + "• Aggressive multi-bar directional momentum

with aligned delta."

mainColor := stkBull ? g1BullColor : g1BearColor

if hasSqz

iconString += I_SQZ

titleString += (titleString == "" ? "" : ", ") + "Delta Squeeze"

descString += (descString == "" ? "" : "\n") + "• Extreme offside aggression trapped by a counter-trend close."

mainColor := sqzBull ? g2BullColor : g2BearColor

if hasExh

iconString += I_EXH

titleString += (titleString == "" ? "" : ", ") + "Exhaustion"

descString += (descString == "" ? "" : "\n") + "• Climactic volume at a session extreme followed by sharp price rejection."

```

mainColor := exhBull ? g2BullColor : g2BearColor

if hasDiv
    iconString += I_DIV
    titleString += (titleString == "" ? "" : ", ") + "Divergence"
    descString += (descString == "" ? "" : "\n") + "• Price sweeps a high/low but violently rejects
with counter-trend delta."
    mainColor := divBull ? g2BullColor : g2BearColor

if hasAbs
    iconString += I_ABS
    titleString += (titleString == "" ? "" : ", ") + (absBull ? "Passive Buyers Absorbing" : "Passive
Sellers Absorbing")
    descString += (descString == "" ? "" : "\n") + "• Massive volume dumped into the tape (High
Effort) with tiny price movement (Low Result)."
    if iconString == I_ABS
        mainColor := absBull ? g2BullColor : g2BearColor

if hasLrg
    iconString += I_LRG
    titleString += (titleString == "" ? "" : ", ") + "Large Body"
    descString += (descString == "" ? "" : "\n") + "• Explosive initiative via historically large
candle body with aligned flow."
    if iconString == I_LRG
        mainColor := isLargeBull ? g1BullColor : g1BearColor

if hasTrp
    iconString += I_TRP
    titleString += (titleString == "" ? "" : ", ") + "Trapped VA"
    descString += (descString == "" ? "" : "\n") + "• Aggressive acceptance and directional drive
gapping away from Value Area."
    if iconString == I_TRP
        mainColor := trappedBull ? g3BullColor : g3BearColor

if hasCvd
    iconString += I_CVD
    titleString += (titleString == "" ? "" : ", ") + "CVD Flip"
    descString += (descString == "" ? "" : "\n") + "• Macro shift in intraday control as Cumulative

```



```

Volume Delta crosses zero."
    if iconString == I_CVD
        mainColor := cvdBull ? g3BullColor : g3BearColor

    string fullTooltip = titleString + "\n" + descString + "\n\n" + baseTip

    float yLoc = isBull ? low : high
    string style = isBull ? label.style_label_up : label.style_label_down

    if array.size(d.sigLabels) > 0
        label lastLbl = array×last(d×sigLabels)
        if label.get_x(lastLbl) == bar_index
            label.set_y(lastLbl, yLoc), label.set_color(lastLbl, mainColor), label.set_style(lastLbl,
style)
            label.set_text(lastLbl, iconString), label.set_tooltip(lastLbl, fullTooltip)
        else
            array.push(d.sigLabels, label.new(bar_index, yLoc, iconString, color=mainColor,
textcolor=color.white, style=style, size=size.small, tooltip=fullTooltip))
        else
            array.push(d.sigLabels, label.new(bar_index, yLoc, iconString, color=mainColor,
textcolor=color.white, style=style, size=size.small, tooltip=fullTooltip))

    while array.size(d.sigLabels) > 50
        label.delete(array.shift(d.sigLabels))
    else
        // Intrabar cleanup for calcOnClose = false
        if array.size(d.sigLabels) > 0 and label.get_x(array×last(d×sigLabels)) == bar_index
            label.delete(array.pop(d.sigLabels))

updateSessionDisplay(SessionDisplay d, color baseC, string name, int stackLvl) =>
    float prevCvd = d.cvd

    d.cPri := close
    d.vSum += cVol
    d.pvSum += hlc3 * cVol
    d×pv2Sum += hlc3 * hlc3 * cVol
    d.cvd += nz(barDelta)

```

```
d.evaluateSignatures(prevCvd, name)
```

```
if not na(d.avgR)
```

```
    float avgHigh = d.xoPri + (d.xavgR / 2.0)
```

```
    float avgLow  = d.xoPri - (d.xavgR / 2.0)
```

```
    if high >= avgHigh
```

```
        d.hitAvgH := true
```

```
    if low <= avgLow
```

```
        d.hitAvgL := true
```

```
if not na(d.stdevR)
```

```
    float sdHigh = d.xoPri + ((d.xstdevR × sdMult) / 2.0)
```

```
    float sdLow  = d.xoPri - ((d.xstdevR × sdMult) / 2.0)
```

```
    if high >= sdHigh
```

```
        d.hitSdH := true
```

```
    if low <= sdLow
```

```
        d.hitSdL := true
```

```
d.hPri := math.max(d.hPri, high)
```

```
d.lPri := math.min(d.lPri, low)
```

```
    color sessionColor = close >= d.oPri ? color.new(color.green, fillTransparency) :  
color.new(color.red, fillTransparency)
```

```
if not na(d.bgBox)
```

```
    box.set_top(d.bgBox, math.max(d.oPri, close))
```

```
    box.set_bottom(d.bgBox, math.min(d.oPri, close))
```

```
    box.set_right(d.bgBox, bar_index)
```

```
    box.set_bgcolor(d.bgBox, sessionColor)
```

```
if drawO and not na(d.oLin)
```

```
    line.set_x2(d.oLin, bar_index)
```

```
if drawH and not na(d.hLin)
```

```
    line.set_x2(d.hLin, bar_index), line.set_y1(d.hLin, d.hPri), line.set_y2(d.hLin, d.hPri)
```

```
if drawL and not na(d.lLin)
```

```
    line.set_x2(d.lLin, bar_index), line.set_y1(d.lLin, d.lPri), line.set_y2(d.lLin, d.lPri)
```

```
if drawC and not na(d.cLin)
```

```
    line.set_x2(d.cLin, bar_index), line.set_y1(d.cLin, close), line.set_y2(d.cLin, close)
```

```

if not na(d.mLin)
    line.set_x2(d.mLin, bar_index), line.set_y1(d.mLin, (d.hPri+d.lPri)*0.5), line.set_y2(d.mLin,
(d.hPri+d.lPri)*0.5)
if not na(d.vLin)
    vVal = d.vSum > 0 ? d.pvSum/d.vSum : close
    line.set_x2(d.vLin, bar_index), line.set_y1(d.vLin, vVal), line.set_y2(d.vLin, vVal)

d.refreshLabels(baseC, name, true, stackLvl)

if alertExpRng and barstate.isconfirmed
    if (expRangeMode == "Average" or expRangeMode == "Both") and not na(d.avgR)
        float avgHigh = d.xoPri + (d.xavgR / 2.0)
        float avgLow = d.xoPri - (d.xavgR / 2.0)
        if high >= avgHigh and prevHigh < avgHigh
            alert("  " + name + ": " + syminfo.ticker + " crossed ABOVE Average Expected Range (" +
+ str.tostring(avgHigh, "#.###") + ")", alert.freq_once_per_bar)
        if low <= avgLow and prevLow > avgLow
            alert("  " + name + ": " + syminfo.ticker + " crossed BELOW Average Expected Range (" +
+ str.tostring(avgLow, "#.###") + ")", alert.freq_once_per_bar)

    if (expRangeMode == "SDev" or expRangeMode == "Both") and not na(d.stdevR)
        float sdHigh = d.xoPri + ((d.xstdevR × sdMult) / 2.0)
        float sdLow = d.xoPri - ((d.xstdevR × sdMult) / 2.0)
        if high >= sdHigh and prevHigh < sdHigh
            alert("  " + name + ": " + syminfo.ticker + " crossed ABOVE SDev Volatility Band (" +
+ str.tostring(sdHigh, "#.###") + ")", alert.freq_once_per_bar)
        if low <= sdLow and prevLow > sdLow
            alert("  " + name + ": " + syminfo.ticker + " crossed BELOW SDev Volatility Band (" +
+ str.tostring(sdLow, "#.###") + ")", alert.freq_once_per_bar)

createSessionDisplay(SessionState state, color baseC, string name) =>
    color opC = color.new(baseC, 0)
    line o = na, line h = na, line l = na, line c = na, line m = na, line v = na, line aH = na, line aL = na,
line sH = na, line sL = na

    color fillC = close >= open ? color.new(color.green, fillTransparency) : color.new(color.red,
fillTransparency)
    box b = box.new(bar_index, open, bar_index, close, border_color = color(na), bgcolor = fillC)

```

```

if drawO
  o := line.new(bar_index, open, bar_index, open, color = opC)
if drawH
  h := line.new(bar_index, high, bar_index, high, color = opC, style = line.style_dashed)
if drawL
  l := line.new(bar_index, low, bar_index, low, color = opC, style = line.style_dashed)
if drawC
  c := line.new(bar_index, close, bar_index, close, color = opC)

if midVwapChoice == "Mid" or midVwapChoice == "Both"
  m := line.new(bar_index, hl2, bar_index, hl2, color = midLineColor, style = line.style_dotted)
if midVwapChoice == "VWAP" or midVwapChoice == "Both"
  v := line.new(bar_index, hlc3, bar_index, hlc3, color = vwapLineColor, style = line.style_dashed)
if expRangeMode == "Average" or expRangeMode == "Both"
  aH := line.new(bar_index, open, bar_index, open, color = avgRangeColor, style =
line.style_dashed)
  aL := line.new(bar_index, open, bar_index, open, color = avgRangeColor, style =
line.style_dashed)
if expRangeMode == "SDev" or expRangeMode == "Both"
  sH := line.new(bar_index, open, bar_index, open, color = sdRangeColor, style =
line.style_dotted, width = 2)
  sL := line.new(bar_index, open, bar_index, open, color = sdRangeColor, style =
line.style_dotted, width = 2)

array<float> f_hR = state.hR.excludeExtremes()
array<float> f_hV = state.hV.excludeExtremes()
array<float> f_hSBody = state.hSBody.excludeExtremes()
array<float> f_hBarRange = state.hBarRange.excludeExtremes()
array<float> f_hBody = state.hBody.excludeExtremes()
array<float> f_hBarVol = state.hBarVol.excludeExtremes()

float aBarRange = array×size(f_hBarRange) > 5 ? array.avg(f_hBarRange) : na
float sBarRange = array×size(f_hBarRange) > 5 ? array.stdev(f_hBarRange) : na
float aBody = array×size(f_hBody) > 5 ? array.avg(f_hBody) : na
float sBody = array×size(f_hBody) > 5 ? array.stdev(f_hBody) : na
float aVol = array×size(f_hBarVol) > 5 ? array.avg(f_hBarVol) : na
float sVol = array×size(f_hBarVol) > 5 ? array.stdev(f_hBarVol) : na

```

```

SessionDisplay.new(
  infoLab = label.new(bar_index, low, "", style = label.style_none, size =
getLSize(labelSizeInput), textalign = text.align_center),
  bgBox = b,
  oLin = o, hLin = h, lLin = l, mLin = m, vLin = v, cLin = c, avgHLin = aH, avgLLin = aL, sdHLin =
sH, sdLLin = sL,
  oPri = open, hPri = high, lPri = low, cPri = close, vSum = cVol, pvSum = hlc3 × cVol, pv2Sum =
hlc3 * hlc3 * cVol, sldx = bar_index,
  avgR = array×size(f_hR) > 0 ? array.avg(f_hR) : na,
  avgV = array×size(f_hV) > 0 ? array.avg(f_hV) : na,
  avgSBody = array×size(f_hSBody) > 0 ? array.avg(f_hSBody) : na,
  stdevR = array×size(f_hR) > 1 ? array.stdev(f_hR) : na,
  stdevV = array×size(f_hV) > 1 ? array.stdev(f_hV) : na,
  stdevSBody = array×size(f_hSBody) > 1 ? array.stdev(f_hSBody) : na,
  avgBarRange = aBarRange,
  sdBarRange = sBarRange,
  avgBody = aBody,
  sdBody = sBody,
  avgBarVol = aVol,
  sdBarVol = sVol,
  cvd = nz(barDelta),
  sigLabels = array.new<label>(),
  hitAvgH = false, hitAvgL = false, hitSdH = false, hitSdL = false
)

```

```

method archiveActive(SessionState this, int keepCount, color baseC, string name, int stackLvl) =>
if not na(this.active)
  array.push(this.hR, this.active.hPri - this.active.lPri)
  if array.size(this.hR) > atrLength
    array.shift(this.hR)
  array.push(this.hV, this.active.vSum)
  if array.size(this.hV) > atrLength
    array.shift(this.hV)
  array.push(this.hCvd, this.active.cvd)
  if array.size(this.hCvd) > atrLength
    array.shift(this.hCvd)
  array.push(this.hSBody, math.abs(this.active.cPri - this.active.oPri))

```

```
if array.size(this.hSBody) > atrLength  
    array.shift(this.hSBody)
```

```
array.push(this.hAvgHits, (this.active.hitAvgH or this.active.hitAvgL) ? 1 : 0)  
if array.size(this.hAvgHits) > atrLength  
    array.shift(this.hAvgHits)
```

```
array.push(this.hSdHits, (this.active.hitSdH or this.active.hitSdL) ? 1 : 0)  
if array.size(this.hSdHits) > atrLength  
    array.shift(this.hSdHits)
```

```
this.active.refreshLabels(baseC, name, false, stackLvl)
```

```
if keepCount > 0  
    array.push(this.history, this.active)  
    while array.size(this.history) > keepCount  
        array.shift(this.history).destroy()  
else  
    this.active.destroy()  
this.active := na
```

```
processSession(SessionState state, bool enabled, string name, string sStr, string sTz, color bC, int  
stackLvl = 0) =>
```

```
    bool inS = enabled and not na(time(timeframe.period, sStr, sTz))
```

```
    if inS
```

```
        if na(state.active)  
            state.active := createSessionDisplay(state, bC, name)  
            state.active.refreshLabels(bC, name, true, stackLvl)  
            state.active.evaluateSignatures(nz(barDelta), name)
```

```
        else
```

```
            updateSessionDisplay(state.active, bC, name, stackLvl)
```

```
    if barstate.isconfirmed
```

```
        array.push(state.hBarRange, high - low)  
        array.push(state.hBody, math.abs(close - open))  
        array.push(state.hBarVol, cVol)
```

```
    if array.size(state.hBody) > 500
```

```

        array.shift(state.hBarRange)
        array.shift(state.hBody)
        array.shift(state.hBarVol)
    else
        if not na(state.active)
            state.archiveActive(maxHistorySessions, bC, name, stackLvl)
        true

//-----
// Execution Logic
//-----

if barstate.isfirst and not timeframe.isintraday
    runtime.error("Intraday only.")

var s1 = initSessionState()
var s2 = initSessionState()
var s3 = initSessionState()
var s4 = initSessionState()
var master = initSessionState()

// --- Gap Processor ---
if isNewDay
    float pClose = close[1]
    float cOpen = open
    float gSize = cOpen - pClose
    float absGSize = math.abs(gSize)

    float aGap = array.size(gapHistory) > 0 ? array.avg(gapHistory) : na
    float sGap = array.size(gapHistory) > 1 ? array.stdev(gapHistory) : na

    array.push(gapHistory, absGSize)
    if array.size(gapHistory) > atrLength
        array.shift(gapHistory)

    globalAvgGap := aGap
    globalTodayGap := absGSize

    currentDayGapIcon := gSize >= 0 ? "📈" : "📉"

```

```

int absGSizeTicks = toTicks(absGSize)

string gapMultStr = not na(aGap) and aGap > 0 ? " (" + fN(absGSize / aGap) + "x Avg)" : ""
float gapZScore = getZ(absGSize, aGap, sGap)

currentDayGapTooltip := "\n\n———— Daily Gap ————\n"
currentDayGapTooltip += "Move: " + (gSize >= 0 ? "+" : "") + fN(gSize)
if not na(pClose) and pClose > 0
    currentDayGapTooltip += fP(gSize/pClose)
currentDayGapTooltip += "\nSize: " + str.toString(absGSizeTicks) + " Ticks" + gapMultStr

if not na(gapZScore)
    currentDayGapTooltip += "\nZ-Score: " + (gapZScore > 0 ? "+" : "") + fN(gapZScore)

if plotGapBoxes
    float midPoint = (pClose + cOpen) / 2
    float gapTop = gSize >= 0 ? cOpen : pClose
    float gapBottom = gSize >= 0 ? pClose : cOpen
    color boxColor = gSize >= 0 ? gapUpColor : gapDnColor
    color bordColor = gSize >= 0 ? gapUpBorder : gapDnBorder

    array.push(gapBoxes, box.new(left = bar_index - 1, top = gapTop, right = bar_index +
gapDuration, bottom = gapBottom, border_color = bordColor, bgcolor = boxColor))
    while array.size(gapBoxes) > maxHistorySessions
        box.delete(array.shift(gapBoxes))

    if showGapMid
        array.push(gapLines, line.new(x1 = bar_index - 1, y1 = midPoint, x2 = bar_index +
gapDuration, y2 = midPoint, color = gapMidLineCol, style = line.style_dotted))
        while array.size(gapLines) > maxHistorySessions
            line.delete(array.shift(gapLines))

// --- Standard Session Processor ---
if modelInput == "Intraday"
    processSession(s1, shS1, nS1, tS1, tzS1, cS1, 0)
    processSession(s2, shS2, nS2, tS2, tzS2, cS2, 1)
    processSession(s3, shS3, nS3, tS3, tzS3, cS3, 2)

```



```

    processSession(s4, shS4, nS4, tS4, tzS4, cS4, 3)
else if modelInput == "Full Day"
    if isNewDay and not na(master.active)
        master.archiveActive(maxHistorySessions, globColor, globName, 0)
    processSession(master, true, globName, "0000-0000:1234567", syminfo.timezone, globColor, 0)
else if modelInput == "ETH"
    processSession(master, true, globName, ethTime, syminfo.timezone, globColor, 0)
else if modelInput == "RTH"
    processSession(master, true, globName, rthTime, syminfo.timezone, globColor, 0)
else if modelInput == "Both"
    processSession(s1, true, "ETH", ethTime, syminfo.timezone, globColor, 0)
    processSession(s2, true, "RTH", rthTime, syminfo.timezone, globColor, 1)

```

```

//-----
// Footprint Data Validation
//-----
if barstate.islast and na(barFootprint) and timeframe.isintraday
    var table fpWarnTable = table.new(position.bottom_right, 1, 1)
    table.cell(fpWarnTable, 0, 0, " ⚠ Footprint Data Unavailable\nSignatures Disabled",
text_color=color×white, bgcolor=color.new(color.red, 20), text_size=size.small)

```

```

//-----
// Summary Table Dashboard
//-----
method drawTableColumns(table t, int col, string name, color c, SessionState st, string gapHist,
string gapLive) =>
    [sR, sRPct, sV, bR, bB, bV, sCvd, sCvdPct, hAvgHits, hSdHits] = st.getAverages()
    [lsR, lsRPct, lsV, lbR, lbB, lbV, lsCvd, lsCvdPct, lAvgHits, lSdHits] = st.getLiveStats()

    t.add(col, 0, name + " Hist.", c)
    t.add(col, 1, sR)
    t.add(col, 2, sRPct)
    t.add(col, 3, sV)
    t.add(col, 4, sCvd)
    t.add(col, 5, sCvdPct)
    t.add(col, 6, bR)
    t.add(col, 7, bB)
    t.add(col, 8, bV)

```

```
t.add(col, 9, gapHist)
t.add(col, 10, hAvgHits)
t.add(col, 11, hSdHits)
```

```
int liveCol = col + 1
t.add(liveCol, 0, name + " Live", color.new(c, 30))
t.add(liveCol, 1, lsR)
t.add(liveCol, 2, lsRPct)
t.add(liveCol, 3, lsV)
t.add(liveCol, 4, lsCvd)
t.add(liveCol, 5, lsCvdPct)
t.add(liveCol, 6, lbR)
t.add(liveCol, 7, lbB)
t.add(liveCol, 8, lbV)
t.add(liveCol, 9, gapLive)
t.add(liveCol, 10, lAvgHits)
t.add(liveCol, 11, lSdHits)
```

```
liveCol + 1
```

```
var table summaryTable = table.new(position.top_right, 15, 12, bgcolor = color.new(color.black, 40),
frame_color = color.new(color.gray, 80), frame_width = 1, border_color = color.new(color.gray,
80), border_width = 1)
```

```
if barstate.islast and showSummaryTable
    table.clear(summaryTable, 0, 0, 14, 11)
```

```
string gapHistStr = not na(globalAvgGap) ? str.tostring(toTicks(globalAvgGap)) : "-"
string gapLiveStr = not na(globalTodayGap) ? str.tostring(toTicks(globalTodayGap)) : "-"
```

```
summaryTable.add(0, 0, "Metrics", color.gray, text.align_left)
summaryTable.add(0, 1, "Session Range (Ticks)", color.gray, text.align_left)
summaryTable.add(0, 2, "Session Range (%)", color.gray, text.align_left)
summaryTable.add(0, 3, "Session Volume", color.gray, text.align_left)
summaryTable.add(0, 4, "Session CVD", color.gray, text.align_left)
summaryTable.add(0, 5, "Session CVD (%)", color.gray, text.align_left)
summaryTable.add(0, 6, "Bar Range (Ticks)", color.gray, text.align_left)
summaryTable.add(0, 7, "Bar Body (Ticks)", color.gray, text.align_left)
```

```
summaryTable.add(0, 8, "Bar Volume", color.gray, text.align_left)
summaryTable.add(0, 9, "Daily Gap (Ticks)", color.gray, text.align_left)
summaryTable.add(0, 10, "Avg Range Hit Rate", color.gray, text.align_left)
summaryTable.add(0, 11, "SDev Range Hit Rate", color.gray, text.align_left)
```

```
int col = 1
```

```
if modelInput == "Intraday"
```

```
    if shS1
```

```
        col := summaryTable.drawTableColumns(col, nS1, cS1, s1, gapHistStr, gapLiveStr)
```

```
    if shS2
```

```
        col := summaryTable.drawTableColumns(col, nS2, cS2, s2, gapHistStr, gapLiveStr)
```

```
    if shS3
```

```
        col := summaryTable.drawTableColumns(col, nS3, cS3, s3, gapHistStr, gapLiveStr)
```

```
    if shS4
```

```
        col := summaryTable.drawTableColumns(col, nS4, cS4, s4, gapHistStr, gapLiveStr)
```

```
else if modelInput == "Both"
```

```
    col := summaryTable.drawTableColumns(col, "ETH", globColor, s1, gapHistStr, gapLiveStr)
```

```
    col := summaryTable.drawTableColumns(col, "RTH", globColor, s2, gapHistStr, gapLiveStr)
```

```
else
```

```
    col := summaryTable.drawTableColumns(col, globName, globColor, master, gapHistStr,
gapLiveStr)
```